

SQL interview questions

1. Question: Write a SQL query to select all columns from a table named "employees".

Solution:

```
SELECT * FROM employees;
```

Explanation: The asterisk (*) is a wildcard that selects all columns. This query will return all rows and all columns from the "employees" table.

2. Question: How would you select only the "first_name" and "last_name" columns from the "employees" table?

Solution:

```
SELECT first_name, last_name FROM employees;
```

Explanation: By specifying the column names separated by commas, we select only those columns. This query will return all rows, but only the "first_name" and "last_name" columns.

3. Question: Write a query to select all employees from the "employees" table where the "salary" is greater than 50000.

Solution:

```
SELECT * FROM employees WHERE salary > 50000;
```

Explanation: The WHERE clause filters the results. This query returns all columns for employees whose salary is greater than 50000.

4. Question: How would you select distinct department names from the "employees" table?

Solution:

```
SELECT DISTINCT department FROM employees;
```

Explanation: The DISTINCT keyword removes duplicate values from the result set. This query will return a list of unique department names.

5. Question: Write a query to select employees whose first name starts with 'A'.

Solution:

```
SELECT * FROM employees WHERE first_name LIKE 'A%';
```

Explanation: The LIKE operator with '%' wildcard is used for pattern matching. 'A%' means "starts with A followed by any number of characters".

6. Question: How would you select employees with a salary between 40000 and 60000?

Solution:

```
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 60000;
```

Explanation: The BETWEEN operator selects values within a given range, inclusive of the boundaries.

7. Question: Write a query to select employees from the IT or HR department.

Solution:

```
SELECT * FROM employees WHERE department IN ('IT', 'HR');
```

Explanation: The IN operator allows you to specify multiple values in a WHERE clause. This query selects employees whose department is either IT or HR.

8. Question: How would you select the top 5 highest paid employees?

Solution:

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

Explanation: ORDER BY sorts the results (DESC for descending order), and LIMIT restricts the number of rows returned.

9. Question: Write a query to select employees whose last name ends with 'son' and have a salary greater than 45000.

Solution:

```
SELECT * FROM employees WHERE last_name LIKE '%son' AND salary > 45000;
```

Explanation: This query combines the LIKE operator for pattern matching with the AND operator to apply multiple conditions.

10. Question: How would you select employees and order them by department ascending and then by salary descending?

Solution:

```
SELECT * FROM employees ORDER BY department ASC, salary DESC;
```

Explanation: You can sort by multiple columns by listing them in the ORDER BY clause. This query first sorts by department in ascending order, then by salary in descending order within each department.

11. Question: How would you select employees whose first name is either 'John', 'Jane', or 'Jim'?

Solution:

```
SELECT * FROM employees WHERE first_name IN ('John', 'Jane', 'Jim');
```

Explanation: The IN operator allows you to specify multiple values in a WHERE clause. This query selects all employees whose first name matches any of the given names.

12. Question: Write a query to select all employees except those in the 'Finance' department.

Solution:

```
SELECT * FROM employees WHERE department != 'Finance';
```

Explanation: The != operator means "not equal to". This query selects all employees whose department is not 'Finance'. You could also use <> instead of !=.

13. Question: How would you select employees hired in the year 2022?

Solution:

```
SELECT * FROM employees WHERE YEAR(hire_date) = 2022;
```

Explanation: The YEAR() function extracts the year from a date. This query selects all employees whose hire date is in 2022.

14. Question: Write a query to select the first 10 employees when ordered alphabetically by last name.

Solution:

```
SELECT * FROM employees ORDER BY last_name ASC LIMIT 10;
```

Explanation: This query orders employees by last name in ascending order (ASC) and then limits the result to the first 10 rows.

15. Question: How would you select employees and concatenate their first and last names into a single column?

Solution:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;
```

Explanation: The CONCAT() function joins strings together. This query creates a new column 'full_name' that combines the first and last names with a space in between.

16. Question: Write a query to select employees who don't have a manager (assuming manager_id can be NULL).

Solution:

```
SELECT * FROM employees WHERE manager_id IS NULL;
```

Explanation: The IS NULL operator checks for NULL values. This query selects all employees who don't have a manager assigned.

17. Question: How would you select employees and calculate their years of service?

Solution:

```
SELECT *, YEAR(CURRENT_DATE) - YEAR(hire_date) AS years_of_service FROM employees;
```

Explanation: This query calculates the years of service by subtracting the hire year from the current year. Note that this is a simplified calculation and doesn't account for specific hire dates within the year.

18. Question: Write a query to select employees and categorize their salaries as 'Low', 'Medium', or 'High'.

Solution:

```
SELECT *,
       CASE
         WHEN salary < 40000 THEN 'Low'
         WHEN salary BETWEEN 40000 AND 80000 THEN 'Medium'
         ELSE 'High'
       END AS salary_category
FROM employees;
```

Explanation: This query uses a CASE statement to categorize salaries into 'Low', 'Medium', or 'High' based on specified ranges.

19. Question: How would you select the top 5% of employees based on salary?

Solution:

```
SELECT * FROM employees
ORDER BY salary DESC
LIMIT (SELECT CEIL(0.05 * COUNT(*)) FROM employees);
```

Explanation: This query first calculates 5% of the total number of employees, rounds up to the nearest integer, and uses that as the LIMIT. The employees are ordered by salary in descending order to get the top earners.

20. Question: Write a query to select employees hired in the last 30 days.

Solution:

```
SELECT * FROM employees
WHERE hire_date >= CURRENT_DATE - INTERVAL 30 DAY;
```

Explanation: This query selects employees whose hire date is within the last 30 days from the current date.

21. Question: How would you select employees and display their salary in thousands (e.g., 50000 as 50K)?

Solution:

```
SELECT first_name, last_name,  
       CONCAT(FLOOR(salary/1000), 'K') AS salary_in_thousands  
FROM employees;
```

Explanation: This query divides the salary by 1000, rounds down to the nearest integer, and concatenates 'K' to represent thousands.

22. Question: Write a query to select employees whose names contain the letter 'e' (case insensitive).

Solution:

```
SELECT * FROM employees  
WHERE LOWER(first_name) LIKE '%e%' OR LOWER(last_name) LIKE '%e%';
```

Explanation: This query uses the LOWER() function to convert names to lowercase, then uses LIKE with wildcards to find names containing 'e'.

23. Question: How would you select the second highest salary from the employees table?

Solution:

```
SELECT DISTINCT salary  
FROM employees  
ORDER BY salary DESC  
LIMIT 1 OFFSET 1;
```

Explanation: This query orders salaries in descending order, skips the first result (OFFSET 1), and then selects the next highest salary (LIMIT 1).

24. Question: Write a query to select employees and calculate their daily rate (assuming 260 working days per year).

Solution:

```
SELECT *, ROUND(salary / 260, 2) AS daily_rate  
FROM employees;
```

Explanation: This query divides the annual salary by 260 to get the daily rate, rounded to 2 decimal places.

25. Question: How would you select employees whose first name and last name start with the same letter?

Solution:

```
SELECT * FROM employees  
WHERE LEFT(first_name, 1) = LEFT(last_name, 1);
```

Explanation: This query uses the LEFT() function to compare the first character of the first name and last name.